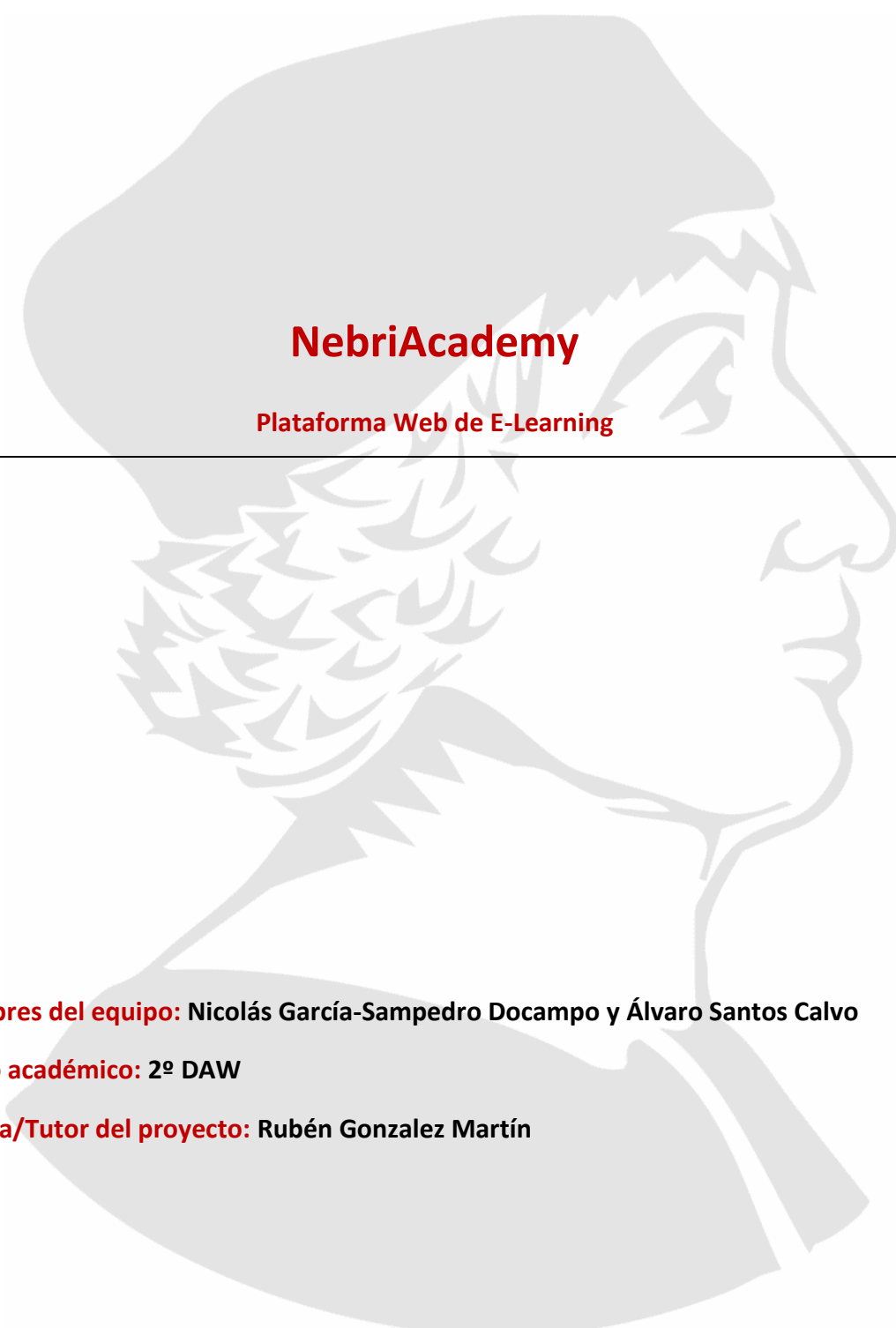




NebriAcademy

Plataforma Web de E-Learning

Nombres del equipo: Nicolás García-Sampedro Docampo y Álvaro Santos Calvo

Curso académico: 2º DAW

Tutora/Tutor del proyecto: Rubén Gonzalez Martín

ÍNDICE PAGINADO

JUSTIFICACIÓN DE PROYECTO	3
INTRODUCCIÓN.....	3
OBJETIVOS	4
OBJETIVO GENERAL	4
OBJETIVOS ESPECÍFICOS	4
DESARROLLO	4
ESTRUCTURA DEL PROYECTO:.....	18
1.- Estructura del backend:	19
2.- Estructura del frontend:	19
MATERIALES Y MÉTODOS:	22
RESULTADOS Y ANÁLISIS	36
CONCLUSIONES.....	37
LÍNEAS DE INVESTIGACIÓN FUTURAS	37
BIBLIOGRAFÍA.....	37
ANEXOS.....	38
AGRADECIMIENTOS	38

JUSTIFICACIÓN DEL PROYECTO

El proyecto NebriAcademy se concibe como una plataforma de E-learning, inspirada en servicios como OpenWebinars o Coursera, orientada a la gestión de contenidos educativos, para los distintos usuarios (profesores y alumnos) y las diferentes relaciones académicas. permitiendo la consulta estructurada de información relacionada con cursos, alumnos, profesores, materiales y evaluaciones.

Este proyecto permite aplicar de forma práctica los conocimientos adquiridos en el desarrollo de aplicaciones web: en la parte del backend, utilizando Node.js, el framework Express y el middleware Multer; para el frontend aplicando React, la biblioteca de gestión de estado global Zustand; y para la BDD, eligiendo Xampp de phpMyAdmin.

INTRODUCCIÓN

NebriAcademy es una aplicación web de E-learning, cuyo objetivo es facilitar el acceso a los cursos online de formación a los estudiantes.

Al tratarse de una plataforma de la Universidad Nebrija, hemos diseñado un sistema que permite a sus alumnos el acceso gratuito, permitiendo a su vez el pago a los alumnos ajenos a esta institución.

Por otra parte, los profesores, antes de poder impartir los cursos, deberán solicitar el acceso mediante un sistema de verificación previo.

Una vez creada la cuenta, un profesor puede crear tantos cursos como desee y subir a estos distintos contenidos (vídeos, apuntes, ejercicios). También tendrá distintas funcionalidades como acceder y evaluar los ejercicios subidos por los alumnos.

Por su parte, los alumnos tienen acceso a los distintos contenidos que suba cada profesor a sus cursos, para ellos disponen de funcionalidades como acceder a una página que contiene todos los cursos de la plataforma y un buscador que les permite identificar los cursos según distintas categorías y niveles, para facilitar la localización del contenido deseado.

Una vez seleccionado el curso, el alumno puede apuntarse o marcarlo como favorito. También puede valorarlo tanto negativamente como positivamente, lo que facilitará el proceso de selección para otros futuros alumnos.

A su vez, el alumno puede hacer comentarios sobre el curso y subir sus propios apuntes para que estén disponibles para el uso por el resto de los alumnos.

También pueden acceder a los ejercicios establecidos por el profesor, cumplimentarlos y entregarlos al profesor para su evaluación.

La página también permite el acceso directo a un apartado denominado “profesores”, donde los alumnos pueden localizar un profesional por su especialización.

Y, por último, en el apartado “apuntes” se muestran disponibles todos los apuntes subidos por los profesores y por los alumnos, con un buscador que

permite seleccionar estos apuntes por categorías y ordenarlos por distintas opciones (populares, novedades, favoritos y mis apuntes).

OBJETIVOS:

OBJETIVO GENERAL

Desarrollar una aplicación web funcional que permita de forma práctica los conocimientos adquiridos en el curso. Hemos elegido una plataforma de E-learning con un backend que exponga la información académica necesaria y un frontend para que los usuarios (profesores y alumnos) puedan acceder de forma sencilla y amigable a los contenidos de los cursos publicados y les permita distintas funcionalidades.

OBJETIVOS ESPECÍFICOS

- Crear modelos para cada entidad de la base de datos.
- Crear una base de datos para implementar los datos, usuarios y archivos.
- Organizar el proyecto mediante una estructura modular.
- Definir rutas específicas para cada recurso académico.
- Permitir la subida de archivos por parte de los usuarios.
- Establecer una interfaz interactiva y amigable para el usuario.
- Distinguir los distintos perfiles de usuarios.
- Incorporar validación de datos.
- Añadir paginación y filtros avanzados.

DESARROLLO

El proyecto lo hemos estructurado en distintas fases:

1.- En primer lugar, para la fase de creación, elegimos la idea para la página web, analizando las distintas plataformas de E-learning existentes en el mercado.

Creamos una idea inicial de página web en Figma:

Login:



NebriAcademy

Escuela de Idiomas Nebrija

¿Eres de la familia Nebrija?

Sí

Estudio actualmente en
un centro asociado a
Nebrija.



No

No estudio actualmente
en un centro asociado a
Nebrija.

Home:

[illegible]

Curso:



HTML5 y CSS3: Maquetación Web

Desarrollo Frontend

Dificultad: Intermedio

Incluso las ilustraciones más llamativas se basan en los conceptos más básicos del diseño. Aprende cómo hacer funcionar el contraste y el color juntos, usando herramientas mínimas para crear mundos visuales potentes que estimulen la imaginación y que conecten con los espectadores.

Petar Posti ha desarrollado una estética única equilibrando múltiples formas y un número limitado de colores que lo ha llevado a trabajar con empresas como Apple, Starbucks y Neapress, así como con publicaciones como Monocle, The Guardian y The New York Times.

**Luis Miguel Richart**
Profesor de Desarrollo Frontend especializado en HTML5 y CSS3.
[Contacto](#)

Introducción: Primeros pasos en HTML5

Contenidos: 3 Lecciones 26 min

[Presentación](#)
[Primeros pasos con el lenguaje](#)
[Mi primera página Web](#)

Capítulo

Capítulo

Capítulo

Capítulo

98% valoraciones positivas (207)

3 horas y 26 minutos

45 Lecciones totales

897 Estudiantes

Valoraciones de estudiantes

☐

Nombre Apellido

Valoración sobre la formación de forma textual.

☐

Nombre Apellido

Valoración sobre la formación de forma textual.

☐

Nombre Apellido

Valoración sobre la formación de forma textual.

☐

Nombre Apellido

Valoración sobre la formación de forma textual.

Mi espacio:

Cursos guardados >

**HTML5 y CSS3: Maquetación Web**

74%

**HTML5 y CSS3: Maquetación Web**

74%

**HTML5 y CSS3: Maquetación Web**

74%

Cursos guardados >

**HTML5 y CSS3: Maquetación Web**

74%

**HTML5 y CSS3: Maquetación Web**

74%

**HTML5 y CSS3: Maquetación Web**

74%

**HTML5 y CSS3: Maquetación Web**

74%

Cursos recomendados >

**HTML5 y CSS3: Maquetación Web**

74%

**HTML5 y CSS3: Maquetación Web**

74%

**HTML5 y CSS3: Maquetación Web**

74%

**HTML5 y CSS3: Maquetación Web**

74%

Mi perfil:

**[Nombre]**
[Correo Electrónico]

Mis datos

Ajustes

Privacidad

Nombre

Apellidos

Correo Electrónico

Teléfono

Guardar cambios

6

Si bien, decidimos modificar esta idea inicial elaborando un diseño más atractivo para el usuario. El resultado del diseño elegido es el siguiente:

Login:

 NebriAcademy

[Iniciar Sesión](#)
[Crear cuenta](#)



Register:

 NebriAcademy

¿Eres estudiante de la familia Nebrija?


Si
Estudio actualmente en un centro asociado a Nebrija


No
No estudio actualmente en un centro asociado a Nebrija

 Soy profesor

 NebriAcademy

Regístrate

Seleccione un país

Seleccione una localidad

[Registrarse](#)

¿Ya tienes cuenta? [inicia sesión](#)

Precio

Por solo 5.99€/mes.

Acceso completo a nuestra plataforma educativa:

- Acceso ilimitado a todos los cursos disponibles
- Materiales de estudio exclusivos y actualizados
- Videos tutoriales y clases grabadas
- Ejercicios prácticos con retroalimentación
- Apuntes descargables en PDF
- Seguimiento de tu progreso académico
- Soporte y comunicación con profesores
- Certificados de finalización de cursos



7

- Alumnos:

 NebriAcademy

Mi espacioCursosProfesoresApuntes

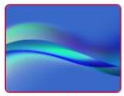
Buscar

Bienvenido/a: Nico Samp

Novedades



Prueba
Nivel: Intermedio
1



Iniciación a las BDD
Nivel: Básico
1

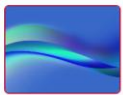


Diseño Responsive
Nivel: Intermedio
0



Email Marketing
Nivel: Intermedio
0

Tus cursos



Iniciación a las BDD
Nivel: Básico
1



Diseño Responsive
Nivel: Intermedio
0

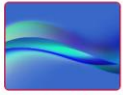


Machine Learning con Python
Nivel: Avanzado
0

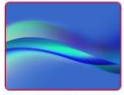
Categorías

ProgramaciónDiseñoCiberseguridadBDDMarketing

Cursos populares



Iniciación a las BDD
Nivel: Básico
1



Python desde Cero
Nivel: Principiante
0



Python Avanzado
Nivel: Avanzado
0



Diseño Web con HTML/CSS
Nivel: Intermedio
0

Política de privacidadNota legalPolítica de cookies



Contactanos: +34 914 521 100
NebriAcademy © 2026

Mi espacio:

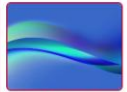
 NebriAcademy

Mi espacioCursosProfesoresApuntes

Buscar...

Tu espacio Nico Samp

Cursos en proceso



Iniciación a las BDD
Nivel: Básico
👍 1



Diseño Responsive
Nivel: Intermedio
👍 0



Machine Learning con Python
Nivel: Avanzado
👍 0

Cursos favoritos



Diseño Web con HTML/CSS
Nivel: Intermedio
👍 0



JavaScript Moderno
Nivel: Intermedio
👍 0



Protección de Datos GDPR
Nivel: Intermedio
👍 0

Tus apuntes



Apuntes python
Estos apuntes tienen los conceptos básicos de python
Nico Samp
Programación
👍 1

Apuntes favoritos



Fundamentos de las BDD
Introducción a las BDD. Autor: © Santiago Fari
Arturo Arizne
BDD
👍 1



Prueba alumno
Sofía Gómez Ruiz
BDD
👍 1



Apuntes python
Estos apuntes tienen los conceptos básicos de python
Nico Samp
Programación
👍 1

[Política de privacidad](#) [Nota legal](#) [Política de cookies](#)



Contactanos: +34 914 521 100
NebriAcademy © 2026

Todos los cursos:

 NebriAcademy

Mi espacioCursosProfesoresApuntes

Buscar...

Cursos

Categorías

Todas

Programación

Diseño

Ciberseguridad

BDD

Marketing

Nivel

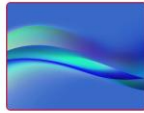
Todos

Básico

Intermedio

Avanzado

Limpiar filtros



Python desde Cero
Categoría: Programación
Nivel: Principiante
Aprende Python desde cero. Curso ideal para iniciarse en la programación.
Profesor: Ana García López
👍 1



Python Avanzado
Categoría: Programación
Nivel: Avanzado
Programación avanzada con Python: decoradores, generadores y patrones de diseño.
Profesor: Ana García López
👍 0



Diseño Web con HTML/CSS
Categoría: Diseño
Nivel: Intermedio
Crea sitios web modernos con HTML5 y CSS3.
Profesor: Miguel Rodríguez Gómez
👍 0



SQL y Bases de Datos
Categoría: BDD
Nivel: Intermedio
Domina SQL y gestiona bases de datos relacionales con MySQL y PostgreSQL.
Profesor: Carlos Martínez Ruiz
👍 0









Curso:

**NebriAcademy**

Mi espacioCursosProfesoresApuntes

Buscar...



Iniciación a las BDD

BDD

Nivel: Básico

Valoración:  2 

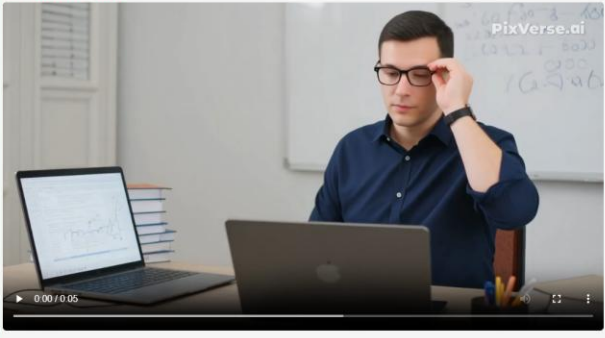
 Favorito

 Apuntarme

Contenido del curso

Videos

Presentación del curso



Apuntes

Apuntes profesor

Fundamentos de las BDD

Introducción a las BDD, Autor: © Santiago Faci
Arturo Arturez

 1

Apuntes alumnos

Prueba alumno

Sofía Gómez Ruiz

 1

Ejercicios

Ejercicio_0

Ejercicio para que uséis lo aprendido en el curso



Comentarios

Sofía Gómez Ruiz

El vídeo me ha gustado mucho 🍷

Editar

Borrar

Comenta...

Enviar

[Política de privacidad](#) [Nota legal](#) [Política de cookies](#)

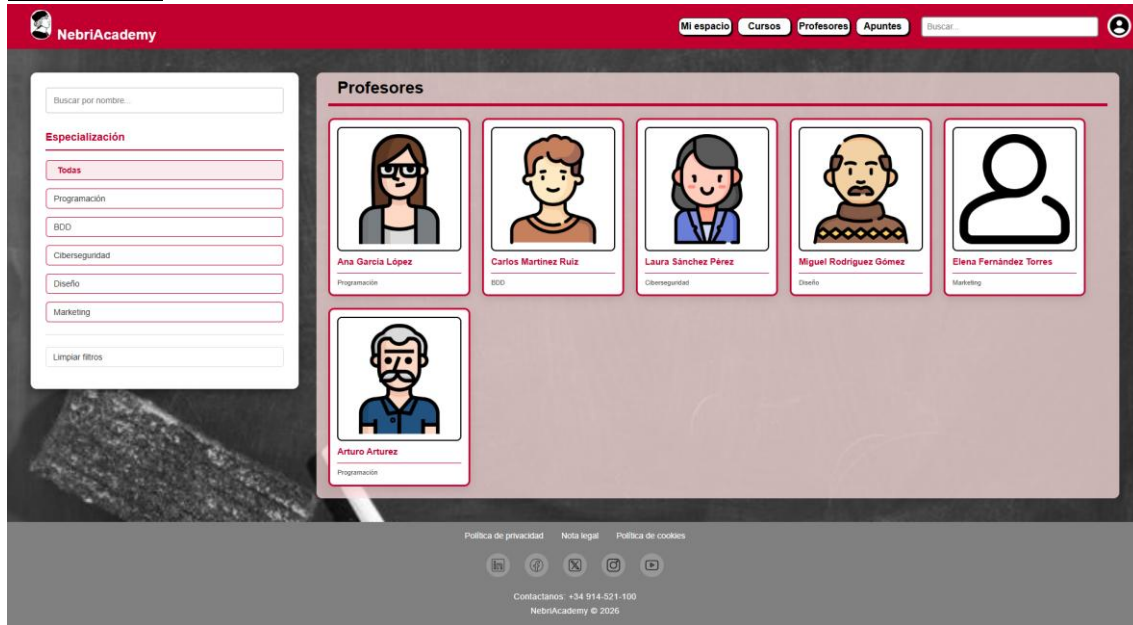
    

Contactanos: +34 914-521-100

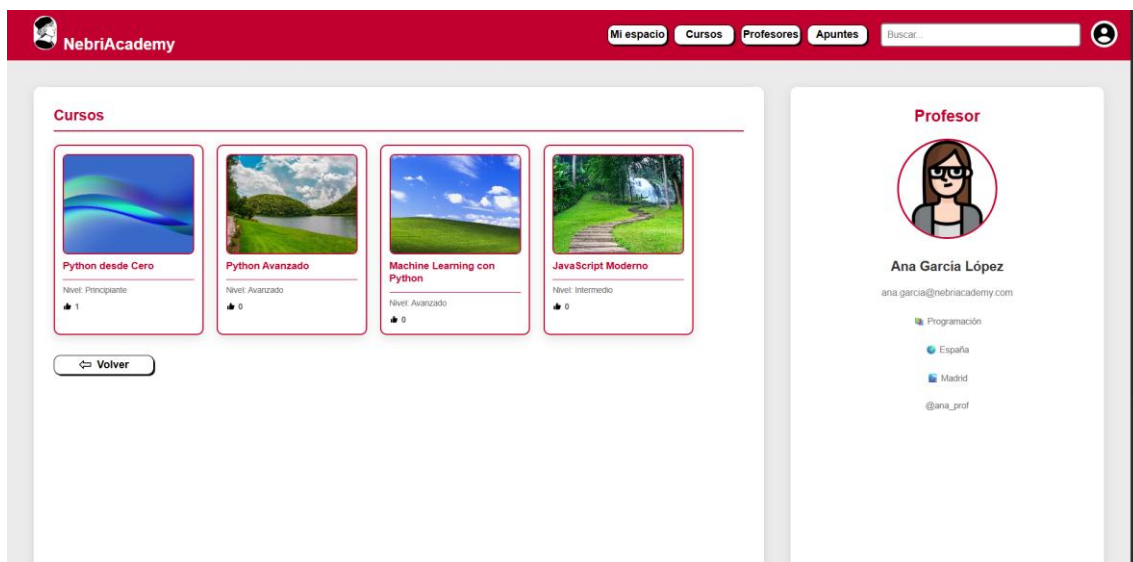
NebriAcademy © 2026



Profesores:



Profesor individual:



Añadir curso:

 NebriAcademy

Apuntes

Añadir curso



Crear Curso

Nombre del curso *

Categoría *

Selecciona categoría

Nivel *

Selecciona nivel

Descripción *

Añadir Apunte (Opcional)

Nombre

Archivo (.pdf, .doc, .docx, .ppt, .pptx)

Seleccionar archivo | Ningún archivo seleccionado

Descripción

Añadir Video (Opcional)

Nombre

Archivo (Video)

Seleccionar archivo | Ningún archivo seleccionado

Añadir Ejercicio (Opcional)

Nombre

Archivo (.zip, .pdf...)

Seleccionar archivo | Ningún archivo seleccionado

Descripción

Selecciona una imagen de fondo



Crear curso

Volver

[Política de privacidad](#) [Nota legal](#) [Política de cookies](#)



Contactanos: +34 914 521-100
NebriAcademy © 2026

Apuntes:

 NebriAcademy

Mi espacio

Cursos

Profesores

Apuntes

Buscar



Buscar...

Categorías

Todas

Programación

Diseño

Ciberseguridad

BDD

Marketing

Mis apuntes

Populares

Novedades

Favoritos

Limpiar filtros

Apuntes

Fundamentos de las BDD
Introducción a las BDD. Autor: © Santiago Faci
Arturo Arturez

BDD

1

Prueba alumno
Sofía Gómez Ruiz

BDD

1

Apuntes python
Estos apuntes tienen los conceptos básicos de python
Nico Samp

Programación

1



Mi perfil:

**NebriAcademy**

Mi espacioCursosProfesoresApuntes

Buscar...



Mi Perfil



Nico Samp
nico@example.com

ALUMNO

 España

 Barcelona

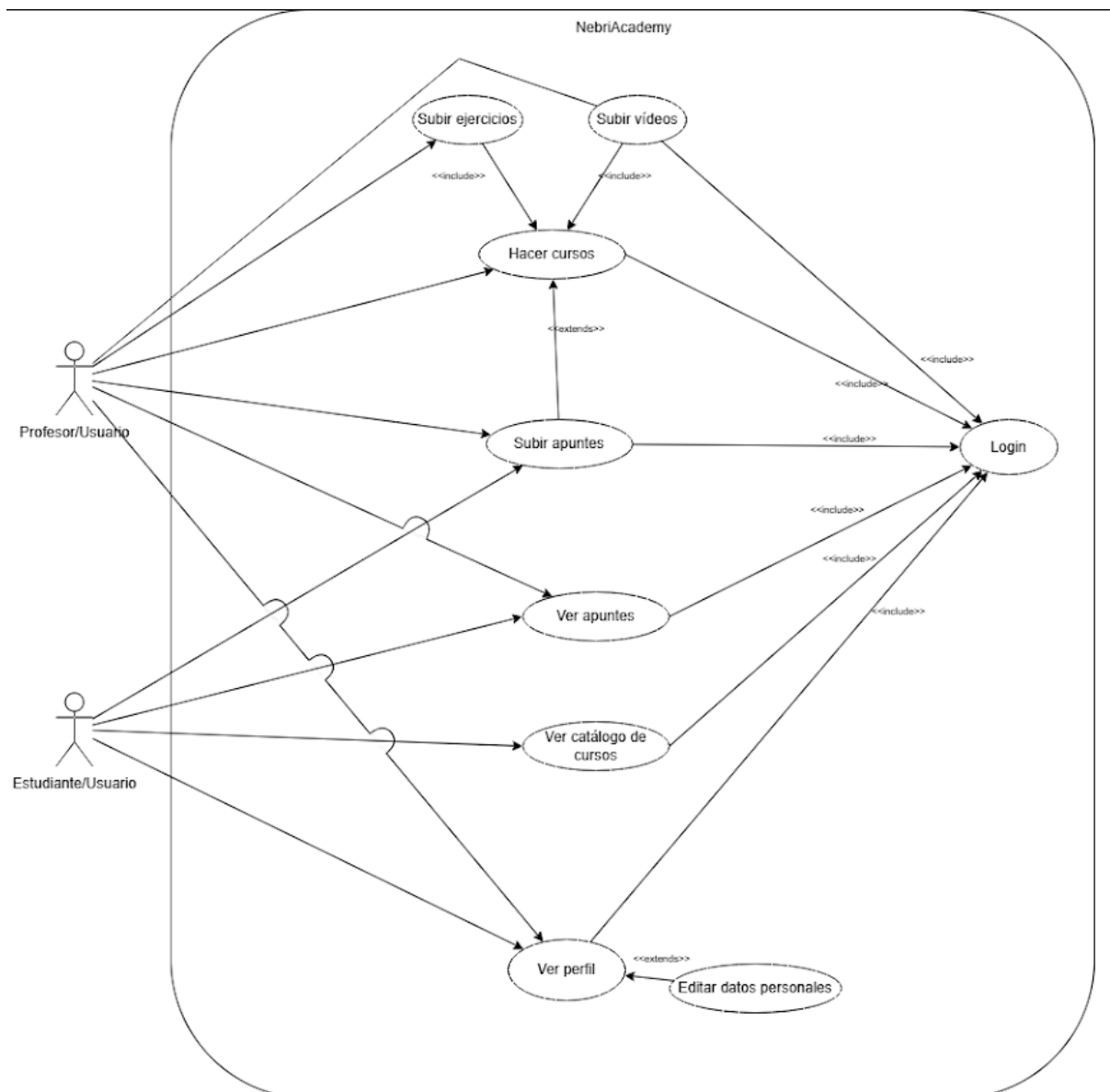
Editar Perfil

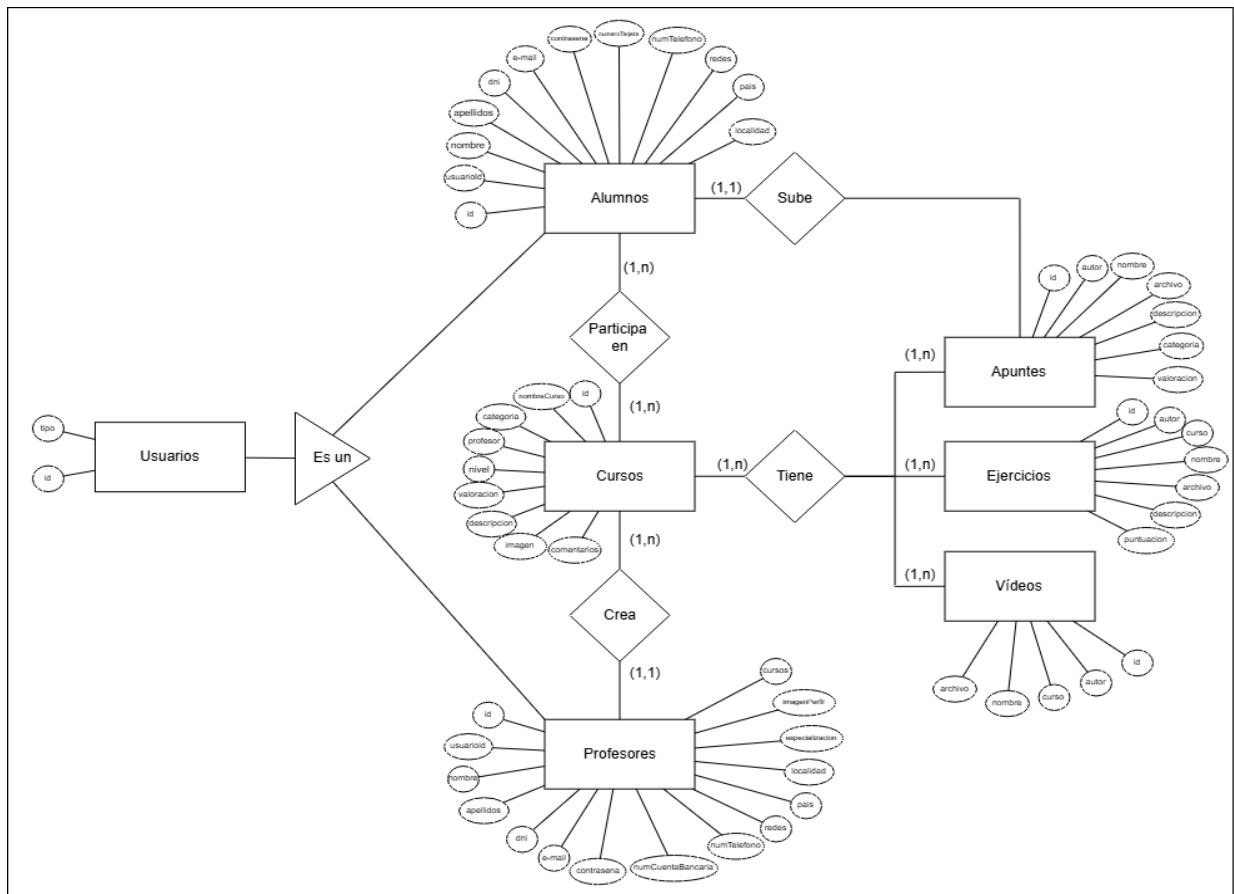
Nombre:	Nico	Apellidos:	Samp
Contraseña:	*****	Número de Tarjeta:	488888488888448
Teléfono:	Tu teléfono	Redes Sociales:	@usuario
País:	España	Localidad:	Barcelona

Guardar Cambios

Volver

2.- En segundo lugar, hemos realizado la toma de requisitos. Elaborando los diagramas de casos de uso y entidad-relación:





3.- Para la base de datos hemos elegido la aplicación XAMPP de phpMyAdmin, que nos permite implementar y tener acceso a todos los datos, usuarios, archivos, etc.

Tabla	Acción	Filas	Tipo	Cotejamiento	Tamaño	Residuo a depurar
☐ alumnos	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	9	InnoDB	utf8mb4_general_ci	80.0 KB	-
☐ apuntes	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	2	InnoDB	utf8mb4_general_ci	48.0 KB	-
☐ apuntesalumnos	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	3	InnoDB	utf8mb4_general_ci	48.0 KB	-
☐ comentarioalumnocurso	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	9	InnoDB	utf8mb4_general_ci	48.0 KB	-
☐ cursos	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	22	InnoDB	utf8mb4_general_ci	32.0 KB	-
☐ cursosalumnos	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	30	InnoDB	utf8mb4_general_ci	48.0 KB	-
☐ ejercicios	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	1	InnoDB	utf8mb4_general_ci	48.0 KB	-
☐ ejerciciosalumnos	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	1	InnoDB	utf8mb4_general_ci	48.0 KB	-
☐ profesores	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	6	InnoDB	utf8mb4_general_ci	80.0 KB	-
☐ profesorescursos	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	22	InnoDB	utf8mb4_general_ci	48.0 KB	-
☐ puntuacionesejercicios	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	1	InnoDB	utf8mb4_general_ci	48.0 KB	-
☐ usuarios	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	16	InnoDB	utf8mb4_general_ci	16.0 KB	-
☐ videos	★ Examinar Estructura Buscar Insertar Vaciar Eliminar	1	InnoDB	utf8mb4_general_ci	48.0 KB	-

4.- A continuación, hemos implementado el **backend**. Hemos aplicado los conocimientos de Node.js y Express, para hacer una API REST, así como una correcta organización del código, mediante una arquitectura modular.

Hemos seguido los principios básicos de una estructura modular para organizar nuestro proyecto:

- **Independencia:** Hemos escrito el código de manera que se puedan hacer las modificaciones que hagan falta sin tener que modificar la aplicación completa.
- **Reutilización:** Hemos trabajado con componentes, lo que nos permite utilizarlos en diferentes partes de la aplicación. Para ahorrar tiempo y espacio.
- **Encapsulamiento:** Hemos separado la lógica del backend, los componentes del frontend y los estilos de las páginas para la organización del código.
- **Escalabilidad:** En el caso de que sea necesario, se podrán implementar nuevas funcionalidades y módulos para otros clientes, sin necesidad de reestructurar el código.

Hemos aplicado la ORM **Sequelize** para operar la base de datos MySQL usando clases JavaScript (modelos) en lugar de escribir SQL puro. Y para conectar Node.js con MySQL, Sequelize usa el driver **MYSQL2**.

Para subir archivos (documentos, imágenes y vídeos) hemos aplicado el middleware **Multer**.

Para la seguridad de la página hemos usado el middleware **CORS**, que bloquea las llamadas desde el frontend a la API.

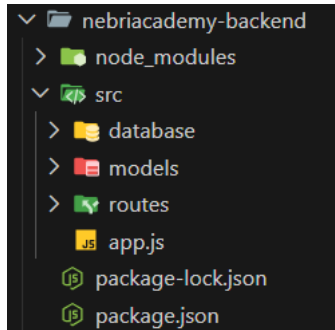
5.- Lo siguiente, ha sido desarrollar el **frontend**: En la parte del cliente, hemos utilizado la librería **React**, que nos ha permitido crear la interfaz interactiva del usuario para que tenga un correcto acceso a todos los contenidos que se ofrecen en la Página Web.

También hemos usado la librería **Zustand** para manejar los distintos perfiles de usuarios, distinguiendo los contenidos accesibles para profesores y alumnos.

○ ESTRUCTURA DEL PROYECTO:

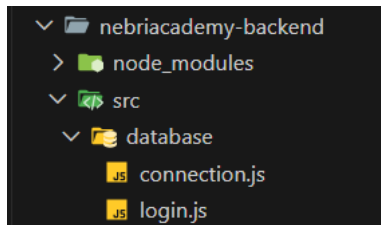
El proyecto sigue una estructura clara y organizada:

1.- Estructura del backend:



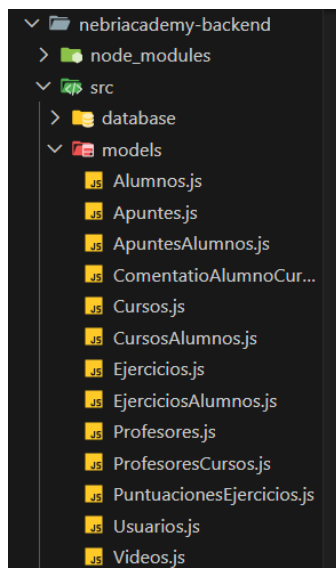
DATABASE:

Gestiona la conexión a la base de datos MySQL mediante Sequelize y centraliza la lógica de autenticación de usuarios (login).



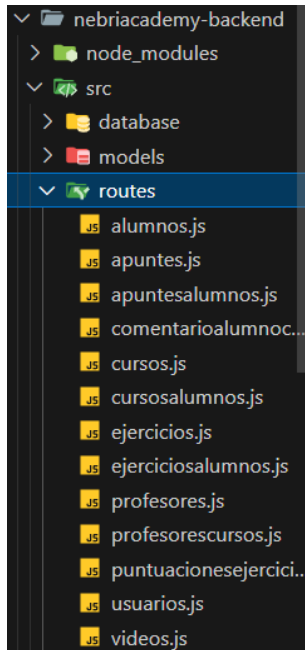
MODELS:

Define la estructura de cada tabla de la base de datos usando Sequelize.

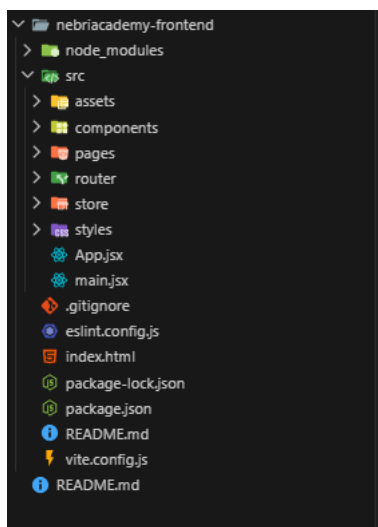


ROUTES:

Contiene los endpoints de la API REST. Cada archivo expone las operaciones disponibles sobre un recurso (crear, leer, actualizar, eliminar) y cualquier lógica adicional como subida de archivos o relaciones entre tablas.

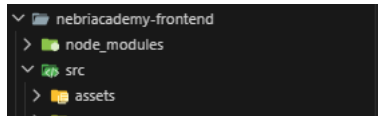


2.- Estructura del frontend:



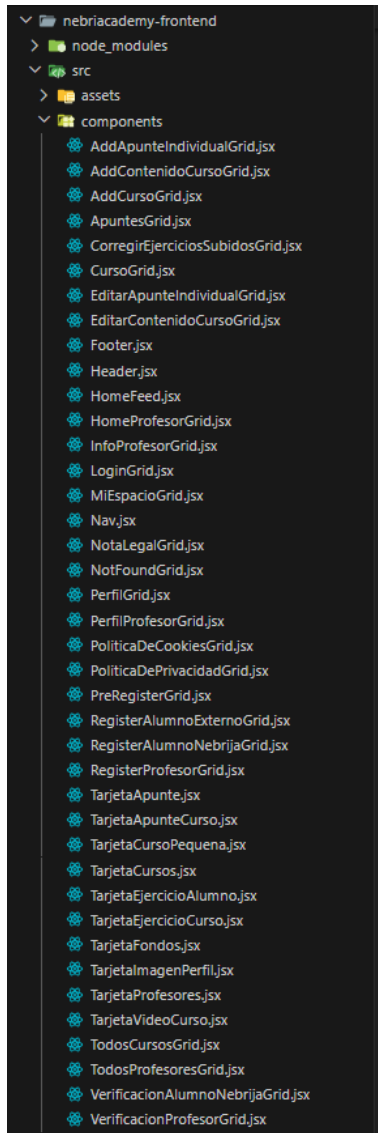
ASSETS:

Almacena todos los recursos estáticos de la aplicación: imágenes, iconos y archivos multimedia que se importan directamente en los componentes. También contiene las subcarpetas donde se guardan los archivos subidos por los usuarios (apuntes, vídeos, imágenes de cursos, fotos de perfil).



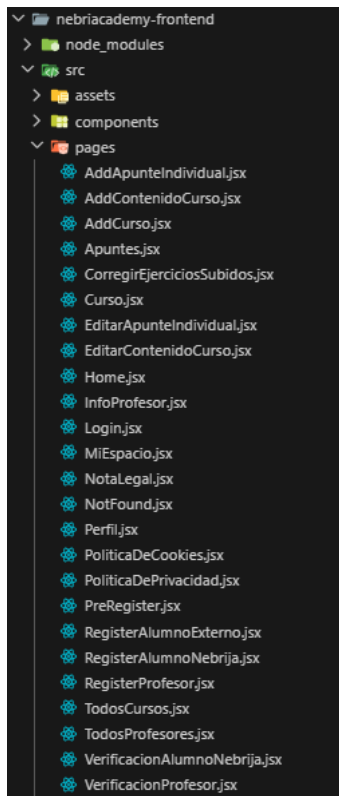
COMPONENTES:

Contiene todos los componentes reutilizables de la interfaz. Aquí reside la lógica principal de cada página.



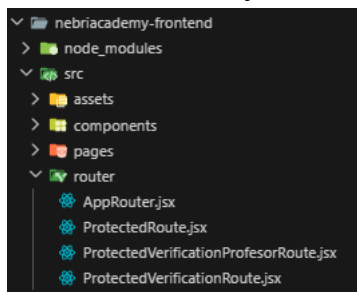
PAGES:

Contiene un archivo por cada página de la aplicación.



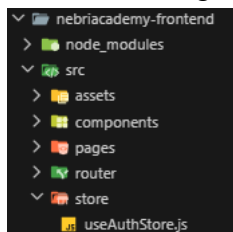
ROUTER:

Configura el sistema de navegación con React Router. Define qué página se muestra en cada URL y protege las rutas privadas: impide el acceso a usuarios no autenticados y restringe ciertas rutas a profesor.



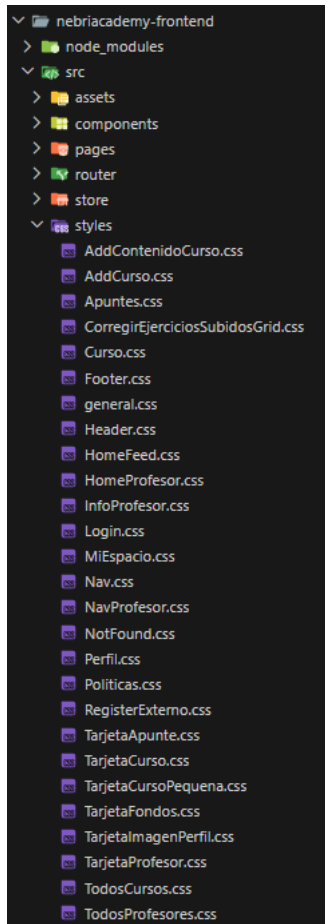
STORE:

Gestiona el estado global de la aplicación con Zustand. Guarda los datos del usuario autenticado (objeto de usuario y tipo: alumno o profesor) y los persiste en localStorage para que la sesión sobreviva a recargas de página.



STYLES:

Contiene los archivos CSS de la aplicación, uno por cada componente o página.



○ MATERIALES Y MÉTODOS:

BACKEND

1. NODE.JS

Qué es:

Entorno de ejecución que permite correr JavaScript fuera del navegador. Es la base sobre la que corre todo el servidor.

Dónde se usa:

Todo el backend se ejecuta en Node.js Los scripts de arranque en package.json son:

ARCHIVO: nebiacademy-backend/package.json

```
{
  > Depurar
  "scripts": {
    "start": "node src/app.js",
    "dev": "nodemon src/app.js"
  },
}
```

2. EXPRESS

Qué es:

Framework HTTP para Node.js. Con Express se crea la API REST, se configuran middlewares y se registran todas las rutas.

Dónde se usa:

ARCHIVO: nebriacademy-backend/src/app.js

```
// Importamos las dependencias principales del servidor
const express = require("express");
const cors = require("cors");
const app = express();
const path = require("path");
const fs = require("fs");

// cors permite peticiones desde el frontend (distinto puerto), express.json() procesa cuerpos JSON
app.use(cors());
app.use(express.json());

// Exponemos cada carpeta de assets como una URL pública para que el frontend pueda descargar los archivos
app.use("/apuntes/files", express.static(path.join(assetsRoot, "Apuntes")));
app.use("/videos/files", express.static(path.join(assetsRoot, "Videos")));
app.use(
  "/ejercicios/files",
  express.static(path.join(assetsRoot, "Ejercicios")),
);
app.use(
  "/ejerciciosalumnos/files",
  express.static(path.join(assetsRoot, "EjerciciosAlumnos")),
);

// Registramos cada módulo de rutas; Express redirige la petición al archivo correspondiente según el prefijo de la URL
app.use("/", require("../routes/index"));
app.use("/alumnos", require("../routes/alumnos"));
app.use("/apuntes", require("../routes/apuntes"));
app.use("/apuntesalumnos", require("../routes/apuntesalumnos"));
app.use("/cursos", require("../routes/cursos"));
app.use("/cursosalumnos", require("../routes/cursosalumnos"));
app.use("/comentarioalumnocurso", require("../routes/comentarioalumnocurso"));
app.use("/ejercicios", require("../routes/ejercicios"));
app.use("/ejerciciosalumnos", require("../routes/ejerciciosalumnos"));
app.use("/profesores", require("../routes/profesores"));
app.use("/profesorescursos", require("../routes/profesorescursos"));
app.use("/puntuacionesejercicios", require("../routes/puntuacionesejercicios"));
app.use("/usuarios", require("../routes/usuarios"));
app.use("/login", require("../database/login"));
app.use("/videos", require("../routes/videos"));

// Middleware global de errores: si cualquier ruta falla de forma inesperada, devuelve un 500 en lugar de romper el servidor
app.use((err, req, res, next) => {
  console.error("Unhandled error:", err && err.stack ? err.stack : err);
  res.status(500).json({
    error: err && err.message ? err.message : "Error interno del servidor",
  });
});
```

Dentro de cada ruta, express.Router() agrupa los endpoints:

ARCHIVO: nebriacademy-backend/src/routes/usuarios.js

```
// — IMPORTACIONES —
const express = require("express");
const router = express.Router();
const Usuarios = require("../models/Usuarios.js");
const Alumnos = require("../models/Alumnos.js");
const Profesores = require("../models/Profesores.js");
```

```
// — GET —
// GET /usuarios – Devuelve todos los registros de la tabla usuarios
router.get("/", async (req, res) => {
  try {
    const all = await Usuarios.findAll();
    res.json({ "Numero de usuarios": all.length, Usuarios: all });
  } catch (e) {
    res.status(500).json({ error: "Server error" });
  }
});

// — POST —
// POST /usuarios – Crea un nuevo registro en la tabla base de usuarios
router.post("/", async (req, res) => {
  // — PUT —
  // PUT /usuarios/:id – Actualiza los datos de un usuario. El tipo en el body determina en qué tabla buscar
  router.put("/:id", async (req, res) => {
    // — DELETE —
    // DELETE /usuarios/:id – Elimina el registro de la tabla usuarios
    router.delete("/:id", async (req, res) => {
```

3. CORS

Qué es:

Middleware que permite peticiones HTTP entre puertos distintos. Sin él, el navegador bloquea las llamadas del frontend (localhost:5173) a la API (localhost:3000).

Dónde se usa:

ARCHIVO: nebriacademy-backend/src/app.js

```
// Importamos las dependencias principales del servidor
const express = require("express");
const cors = require("cors");
const app = express();
const path = require("path");
const fs = require("fs");

// cors permite peticiones desde el frontend (distinto puerto), express.json() procesa cuerpos JSON
app.use(cors());
```

4. SEQUELIZE (ORM)

Qué es:

ORM (Object-Relational Mapper) que permite operar la base de datos MySQL usando clases JavaScript (modelos) en lugar de escribir SQL puro.

Dónde se usa:

ARCHIVO: nebriacademy-backend/src/database/connection.js

```
// Usamos Sequelize como ORM para conectar con la base de datos MySQL
const { Sequelize } = require("sequelize");

// Creamos la instancia de conexión con el nombre de la BD, usuario y contraseña
const sequelize = new Sequelize("nebriacademy", "root", "", {
  host: "localhost",
  dialect: "mysql",
});

// Verificamos que la conexión es correcta al arrancar la aplicación
sequelize
  .authenticate()
  .then(() => {
    console.log("Conexión establecida correctamente.");
  })
  .catch((error) => {
    console.error("No se pudo conectar a la base de datos.", error);
  });

// Exportamos la instancia para que los modelos puedan utilizarla
module.exports = sequelize;
```

ARCHIVO: nebriacademy-backend/src/models/Usuarios.js

```
// — IMPORTACIONES —
// Importamos DataTypes para definir los tipos de columna y la conexión a la BD
const { DataTypes } = require("sequelize");
const sequelize = require("../database/connection");

// — MODELO —
// Definimos el modelo "usuarios", que es la entidad base compartida por alumnos y profesores.
const Usuarios = sequelize.define(
  "usuarios",
  {
    // El campo tipo indica si el registro pertenece a un alumno o un profesor.
    tipo: {
      type: DataTypes.ENUM("alumno", "profesor"),
      allowNull: false,
    },
  },
  { timestamps: false },
);

// — EXPORTAR —
module.exports = Usuarios;
```

Métodos de Sequelize usados en las rutas:

```
const all = await ApuntesAlumnos.findAll();
const alumno = await Alumnos.findPk(req.params.id);
```

```
const nuevo = await Usuarios.create(req.body);
const u = await Model.findByPk(req.params.id);
await u.destroy();
```

5. MYSQL2

Qué es:

Driver de bajo nivel que conecta Node.js con MySQL. Sequelize lo usa internamente; no se llama directamente en el código.

Dónde se usa:

Se declara en package.json y Sequelize lo carga al ver "dialect: mysql" en la configuración de connection.js.

ARCHIVO: nebriacademy-backend/package.json

```
"dependencies": {
  "cors": "^2.8.5",
  "express": "^5.2.1",
  "multer": "^2.0.2",
  "mysql2": "^3.15.3",
```

6. MULTER

Qué es:

Middleware para gestionar subidas de ficheros (multipart/form-data). Guarda los archivos en disco antes de que el handler de la ruta los procesa.

Dónde se usa:

En las rutas: apuntes.js, videos.js, ejercicios.js, ejerciciosalumnos.js.

ARCHIVO: nebriacademy-backend/src/routes/videos.js

```

// — IMPORTACIONES —
const express = require("express");
const router = express.Router();
const multer = require("multer");
const path = require("path");
const fs = require("fs");

const Videos = require("../models/Videos.js");
const Profesores = require("../models/Profesores.js");

// — CONFIGURACIÓN (multer) —
// Carpeta donde se guardan físicamente los archivos de vídeo subidos
const uploadDir = path.join(
  __dirname,
  "../../nebiacademy-frontend/src/assets/Videos",
);

// Configuramos multer para guardar el archivo en uploadDir conservando el nombre original
const upload = multer({
  storage: multer.diskStorage({
    destination: uploadDir,
    filename: (req, file, cb) => cb(null, path.basename(file.originalname)),
  }),
});

// — POST —
// POST /videos – Sube un nuevo vídeo. Solo los profesores pueden hacerlo.
router.post("/", upload.single("archivo"), async (req, res) => {
  try {
    if (!req.file) return res.status(400).json({ error: "Archivo requerido" });
    const { nombre, curso, profileId, tipo } = req.body;

    if (!nombre || !curso || !profileId || !tipo)
      return res.status(400).json({ error: "Datos incompletos" });

    let profesorId = null;

    // Solo aceptamos subidas de vídeo si el usuario es un profesor válido
    if (tipo === "profesor") {
      const p = await Profesores.findByPk(profileId);
      if (p) profesorId = p.id;
    }

    if (!profesorId)
      return res
        .status(400)
        .json({ error: "Profesor no identificado o no autorizado" });

    // Creamos el registro del vídeo en BD con el archivo ya guardado en disco
    const nuevo = await Videos.create({
      autor: profesorId,
      curso,
      nombre,
      archivo: req.file.filename,
      valoracion: 0,
    });

    res.status(201).json({ id: nuevo.id, archivo: nuevo.archivo });
  } catch (e) {
    console.error("Error creando video:", e);
    res.status(500).json({ error: "Error creando video" });
  }
}

```

```
// — PUT —
// PUT /videos/:id - Actualiza los datos de un vídeo. Si se adjunta archivo nuevo, también se actualiza
router.put("/:id", upload.single("archivo"), async (req, res) => {
  try {
    const v = await Videos.findByPk(req.params.id);
    if (!v) return res.status(404).json({ error: "No encontrado" });

    const updates = { ...req.body };
    if (req.file) updates.archivo = req.file.filename;

    const updated = await v.update(updates);
    res.json(updated);
  } catch (e) {
    res.status(500).json({ error: "Server error" });
  }
});
```

7. NODEMON

Qué es:

Herramienta de desarrollo que reinicia el servidor automáticamente al detectar cambios en los archivos del proyecto.

Dónde se usa:

ARCHIVO: `nebriacademy-backend/package.json`

```
{
  "dev": "nodemon src/app.js"
},
```

```
C:\Users\nicog\Desktop\NebriAcademy2\nebriacademy-backend>cd src
C:\Users\nicog\Desktop\NebriAcademy2\nebriacademy-backend\src>nodemon app.js
[nodemon] 3.1.11
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node app.js`
Servidor ejecutándose en http://localhost:3000
Executing (default): SELECT 1+1 AS result
Conexión establecida correctamente.
```

FRONTEND

8. REACT + REACT DOM

Qué es:

React es una librería de UI basada en componentes funcionales.
React DOM es el puente que los renderiza en el HTML real.

Dónde se usa:

ARCHIVO: nebriacademy-frontend/src/main.jsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

ARCHIVO: nebriacademy-frontend/src/App.jsx

```
import AppRouter from "./router/AppRouter.jsx";

import "./styles/general.css";
import "./styles/Header.css";
import "./styles/Nav.css";
import "./styles/NavProfesor.css";
import "./styles/AddCurso.css";
import "./styles/Footer.css";
import "./styles/Login.css";
import "./styles/RegisterExterno.css";
import "./styles/Perfil.css";
import "./styles/TodosCursos.css";
import "./styles/Políticas.css";
import "./styles/Curso.css";
import "./styles/AddContenidoCurso.css";
import "./styles/HomeFeed.css";
import "./styles/Apuntes.css";
import "./styles/TarjetaCursoPequena.css";
import "./styles/TarjetaProfesor.css";
import "./styles/TodosProfesores.css";
import "./styles/MiEspacio.css";
import "./styles/HomeProfesor.css";
import "./styles/NotFound.css";
import "./styles/TarjetaApunte.css";

function App() {
  return <AppRouter />;
}

export default App;
```

Hooks de React usados en todo el proyecto:

- useState → estado local de cada componente

```
// — IMPORTACIONES —
import { useState, useEffect } from "react";
import { useNavigate } from "react-router-dom";
import useAuthStore from "../store/useAuthStore";

// — COMPONENTE —
// Formulario para que cualquier usuario (alumno o profesor) pueda subir un nuevo apunte
function AddApunteIndividualGrid() {
  const navigate = useNavigate();
  const { user: usuario, tipo } = useAuthStore();

  // — ESTADO —
  const [formData, setFormData] = useState({
    nombre: "",
    descripcion: "",
    categoria: "",
  });
  const [file, setFile] = useState(null);
  const [categorias, setCategorias] = useState([]);
  const [error, setError] = useState(null);
  const [loading, setLoading] = useState(false);
```

- useEffect → peticiones a la API al montar el componente

```
// — IMPORTACIONES —
import { useState, useEffect } from "react";
import { useNavigate } from "react-router-dom";
import useAuthStore from "../store/useAuthStore";

// — COMPONENTE —
// Formulario para que cualquier usuario (alumno o profesor) pueda subir un nuevo apunte
function AddApunteIndividualGrid() {
  const navigate = useNavigate();
  const { user: usuario, tipo } = useAuthStore();

  // — ESTADO —
  const [formData, setFormData] = useState({
    nombre: "",
    descripcion: "",
    categoria: "",
  });
  const [file, setFile] = useState(null);
  const [categorias, setCategorias] = useState([]);
  const [error, setError] = useState(null);
  const [loading, setLoading] = useState(false);

  // Cargamos las categorías disponibles al montar el componente para rellenar el selector
  useEffect(() => {
    fetch("http://localhost:3000/apuntes/categorias")
      .then((respuesta) => respuesta.json())
      .then((datos) => {
        setCategorias(Array.isArray(datos.categorias) ? datos.categorias : []),
      })
      .catch((error) => console.error("Error cargando categorias:", error));
  }, []);
```

- useRef → referencias directas al DOM (p. ej. slider)

```
// — IMPORTACIONES —
import { useState, useRef } from "react";
import useAuthStore from "../store/useAuthStore";
import { useNavigate } from "react-router-dom";
import flecha from "../assets/flecha-correcta.png";
import TarjetaFondos from "../TarjetaFondos";

// Referencias a los inputs de tipo file para poder limpiarlos tras el envío exitoso
const fileInputRef = useRef(null);
const fileVideoInputRef = useRef(null);
const fileEjercicioInputRef = useRef(null);
```

9. REACT ROUTER DOM

Qué es:

Gestiona la navegación entre páginas sin recargar el navegador (SPA). Permite rutas públicas, privadas y dinámicas.

Dónde se usa:

ARCHIVO: nebriacademy-frontend/src/router/AppRouter.jsx

```
import { BrowserRouter, Routes, Route } from "react-router-dom";

function AppRouter() {
  return (
    <BrowserRouter>
      <Routes>
        { /* Rutas públicas: accesibles sin estar logueado */ }
        <Route path="/" element={<Login />} />
        <Route path="/PreRegister" element={<PreRegister />} />

        <Route
          path="/Register/RegisterAlumnoExterno"
          element={<RegisterAlumnoExterno />}
        />

        { /* Flujo de registro para alumnos de Nebrija: verificación → formulario de registro */ }
        <Route
          path="/Register/VerificacionAlumnoNebrija"
          element={<VerificacionAlumnoNebrija />}
        />
        <Route
          path="/Register/RegisterAlumnoNebrija"
          element={
            // ProtectedVerificationRoute impide acceder al registro sin haber verificado el email primero
            <ProtectedVerificationRoute>
              <RegisterAlumnoNebrija />
            </ProtectedVerificationRoute>
          }
        />
      </Routes>
    </BrowserRouter>
  );
}
```

```

    /* Rutas exclusivas de profesor: requieren además que el tipo de usuario sea "profesor" */
    <Route
      path="/Home/AddCurso"
      element={
        <ProtectedRoute requiredTipo="profesor">
          <AddCurso />
        </ProtectedRoute>
      }
    />
    <Route
      path="/Home/Cursos/:id/EditarContenidoCurso"
      element={
        <ProtectedRoute requiredTipo="profesor">
          <EditarContenidoCurso />
        </ProtectedRoute>
      }
    />
    <Route
      path="/Home/Cursos/:id/CorregirEjercicios/:id"
      element={
        <ProtectedRoute requiredTipo="profesor">
          <CorregirEjerciciosSubidos />
        </ProtectedRoute>
      }
    />
  </Routes>
</BrowserRouter>
);
}

```

También se usa en componentes mediante hooks:

```

import { useNavigate, useParams } from "react-router-dom";
function ApuntesGrid() {
  const { id } = useParams();
  const navigate = useNavigate();
  /* Botones de editar/borrar visibles solo en modo edición y si el usuario es el autor */
  {editMode && canEdit(ap) && (
    <div className="apunte-edit-controls">
      <button
        onClick={() =>
          navigate(`/Home/Apuntes/EditarApunte/${ap.id}`, {
            state: { apunte: ap },
          })
        }
      />
    </div>
  )}
}

```

10. ZUSTAND

Qué es:

Gestor de estado global ligero. Almacena el usuario logueado y lo persiste en localStorage para que la sesión sobreviva a recargas de página.

Dónde se usa:

ARCHIVO: nebiacademy-frontend/src/store/useAuthStore.js


```
// — IMPORTACIONES —
import { create } from "zustand";
import { persist } from "zustand/middleware";

// — STORE —
// Store global de autenticación usando Zustand.
// "persist" hace que el estado se guarde en localStorage para que sobreviva a recargas de página.
const useAuthStore = create(
  persist(
    (set) => ({
      user: null,
      tipo: null,

      // setUser: guarda el usuario y su tipo (alumno o profesor) al hacer login
      setUser: (userObj, tipo) =>
        set({
          user: userObj,
          tipo: tipo || userObj?.tipo || null,
        }),

      // logout: limpia el usuario y el tipo al cerrar sesión
      logout: () => set({ user: null, tipo: null }),
    }),
    {
      // Nombre de la clave en localStorage donde se persisten los datos
      name: "auth-storage",
      // Solo persistimos user y tipo, no funciones
      partialize: (state) => ({ user: state.user, tipo: state.tipo }),
    }
  )
);

// — EXPORTAR —
export default useAuthStore;
```

Se consume con un selector en cualquier componente:

Ejemplo en HomeFeed.jsx

```
const storeUser = useAuthStore((state) => state.user);
```

Ejemplo en Login.jsx

```
const navigate = useNavigate();
const setUser = useAuthStore((state) => state.setUser);

if (respuesta.ok) {
  // Guardamos el usuario en el store global y navegamos al home
  setUser(datos.usuario, datos.tipo);
  navigate("/Home");
} else {
  setError(datos.error || "Error en el login");
}
```

Ejemplo en Header/Nav

```
const { user: usuario, tipo, logout: logoutStore } = useAuthStore();

// — FUNCIONES —
// Limpia el store de autenticación y redirige al login al cerrar sesión
const handleLogout = () => {
  logoutStore();
  navigate("/");
  setIsdesplegableOpen(false);
};
```

11. REACT-SLICK + SLICK-CAROUSEL

Qué es:

React-slick: componente React que crea carruseles de forma sencilla. slick-carousel: paquete CSS base requerido por react-slick para que los estilos funcionen correctamente.

Dónde se usa:

Archivos: HomeFeed.jsx y MiEspacioGrid.jsx

ARCHIVO: `nebriacademy-frontend/src/components/HomeFeed.jsx`

```
// — IMPORTACIONES —
import { useEffect, useState, useRef } from "react";
import useAuthStore from "../store/useAuthStore";
import { useNavigate } from "react-router-dom";
import TarjetaCursoPequena from "../TarjetaCursoPequena";
import Slider from "react-slick";
import "slick-carousel/slick/slick.css";
import "slick-carousel/slick/slick-theme.css";

// Configuración del carrusel: 4 tarjetas visibles, responsive hasta 2 en móvil
const settingsSlider = {
  dots: false,
  infinite: true,
  speed: 500,
  slidesToShow: 4,
  slidesToScroll: 1,
  responsive: [
    { breakpoint: 1024, settings: { slidesToShow: 3 } },
    { breakpoint: 768, settings: { slidesToShow: 2 } },
  ],
};

// Referencias a cada slider para poder controlarlos manualmente con los botones de flecha
const novedadesSliderRef = useRef(null);
const tusCursosSliderRef = useRef(null);
const popularesSliderRef = useRef(null);
```

```

<button
  className="carousel-btn carousel-btn-left"
  onClick={() => handleSliderArrow(novedadesSliderRef, "left")}
  aria-label="Anterior"
>
  <
</button>
<Slider
  ref={novedadesSliderRef}
  {...settingsSlider}
  className="HomeFeed-novedades-carousel"
>
  {novedades().length > 0 ? (
    novedades().map((curso) => (
      <TarjetaCursoPequena
        key={curso.id}
        name={curso.nombreCurso}
        cursoId={curso.id}
        nivel={curso.nivel}
        valoracion={curso.valoracion || 0}
        imagen={curso.imagen}
      />
    ))
  ) : (
    <p className="mensaje-vacio">No hay cursos disponibles</p>
  )}
</Slider>

```

12. VITE

Qué es:

Bundler moderno y servidor de desarrollo ultrarrápido. Compila el JSX/JS, gestiona módulos ES y aplica Hot Module Replacement (HMR) para ver cambios al instante.

Dónde se usa:

ARCHIVO: `nebriacademy-frontend/vite.config.js`

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
})
```

ARCHIVO: nebriacademy-frontend/package.json

```
"scripts": {
  "dev": "vite",
  "build": "vite build",
  "lint": "eslint .",
  "preview": "vite preview"
},
```

○ RESULTADOS Y ANÁLISIS

Como resultado del proyecto se ha obtenido:

- Backend funcional para una plataforma de E-learning.
- API REST estructurada y modular.
- Endpoints operativos para la consulta de datos académicos.
- Un frontend con una interfaz funcional y de fácil acceso para los distintos perfiles de usuarios.
- Soporte de carga y almacenamiento de archivos.
- Interactividad entre los alumnos y profesores.
- Búsqueda y localización optimizada de los contenidos de la plataforma.

CONCLUSIONES

La plataforma de E-learning NebriAcademy cumple con el objetivo inicial de servir de formación on-line de manera similar a Coursera y OpenWebinars, facilitando la carga de material didáctico a los usuarios profesores y el acceso a los mismos a los usuarios alumnos.

Lo que hemos intentado que sea diferente a las plataformas existentes es su fácil accesibilidad y sencillez de manejo.

El proyecto ha reforzado nuestros conocimientos sobre Node.js, Express, BDD, React y CSS.

Y nos ha hecho buscar e investigar nuevas tecnologías, como CORS, MySQL2, Multer, Zustand, React-Slick y Slick-Carousel.

LÍNEAS DE INVESTIGACIÓN FUTURAS

- A futuro se va a implementar en la aplicación otro perfil de usuario denominado “administrador”. Sus funciones serán
 - Permitir el acceso a nuevos profesores
 - Verificar que los alumnos estén matriculados en alguna institución de Nebrija.
 - Solucionar incidencias que sean reportadas por los alumnos y profesores relativas a problemas de la página web.
- Permitir que el usuario “profesor” tenga acceso a los cursos de otros profesores (ver cursos, valorar o hacer comentarios).
- Optimizar el motor de búsqueda para facilitar la localización de los cursos.
- Trasladar el proyecto al lenguaje Vue para lograr afianzar los conocimientos adquiridos durante el curso.
- Mejorar el responsive (como por ejemplo en el slider)

BIBLIOGRAFÍA

- Documentación oficial de Node.js ⇒ [Index | Node.js v25.2.1 Documentation](#)
- Documentación de Express.js ⇒ [Express routing](#)
- [Flaticon](#) (iconos)
[Flaticon](#)
- [Página de la universidad Nebrija](#)
[Nebrija](#)
- [Página Node](#)
[Node](#)
- [Página Nodemon](#)
[Nodemon](#)

- [Página Multer](#)
[Multer](#)
- [Página Zustand](#)
[Zustand](#)
- [Página MySQL2](#)
[MySQL2](#)
- [Página Cors](#)
[Cors](#)
- [Página Sequelize](#)
[Sequelize](#)

ANEXOS

- Manual de Identidad corporativa de Universidad Nebrija:

[Manual de identidad Nebrija](#)

- Diagramas de casos de uso y de entidad-relación:

[Casos de uso](#)

[Entidad-Relación](#)

AGRADECIMIENTOS

Queríamos agradecer a nuestros profesores Rubén González Martín y Francisco Albiar Jiménez por aportarnos los conocimientos necesarios para este proyecto y habernos guiado durante todo el proceso de creación de este.